

POSTECH FIAP

Arquitetura e Desenvolvimento Java

RELATORIO TECNICO

Tech Challenge - Fase 02

Sistema de Gestao de Restaurantes

Alunos

RM370614 - Vinicius Campos Fanti

RM373960 - Alex Sousa

RM373739 - Lucas Goncalves Prado das Neves

RM372334 - Higo Otaviano da Rocha

RM371904 - Marcos Sanches Polido Junior

Curso

Arquitetura e Desenvolvimento Java

Tecnologia

Spring Boot 3.5 + PostgreSQL 17 + Docker

Data

14/07/2026

1. Descrição da Arquitetura

O projeto foi desenvolvido seguindo os princípios de **Clean Architecture**, organizando o código em camadas concêntricas onde as dependências sempre apontam para o interior — o núcleo de domínio é completamente independente de frameworks, banco de dados ou qualquer detalhe de infraestrutura. Isso garante testabilidade máxima, baixo acoplamento e alta coesão em todas as camadas.

Diagrama de Camadas



Responsabilidade de cada camada

Camada	Pacote	Responsabilidade
*ApiController	infra/web	Recebe requisições HTTP, desserializa JSON, chama Controller do core
Controller	core/controller	Orquestra casos de uso; é agnóstico de Spring
UseCase	core/usecase	Uma operação de negócio por classe
Gateway (interface)	core/gateway	Porta de persistência — implementada na infra
*JpaGateway	infra/database/jpa	Implementa o Gateway via Spring Data JPA
Entidade JPA	infra/database/jpa/entity	Mapeamento objeto-relacional
Domínio	core/domain	Entidades de negócio com invariantes validadas no construtor
InjecaoDependenciaConfiguration	infra/config	Wiring manual dos beans (sem @Service no core)
GlobalExceptionHandler	infra/web/exception	Intercepta exceções e retorna JSON padronizado

Fluxo de uma requisição: *ApiController (infra/web) → Controller (core) → UseCase (core) → Gateway interface (core) → *JpaGateway (infra/database) → JPA Repository → Banco.

As classes de *core* não importam nada de *infra*. A inversão de dependência é feita via **InjecaoDependenciaConfiguration**, que instancia os casos de uso e controllers manualmente, reforçando o desacoplamento do domínio em relação ao framework.

2. Modelagem das Entidades e Relacionamentos

O sistema possui quatro entidades de domínio principais. Cada entidade vive em *core/domain* e é mapeada para JPA somente na camada de infraestrutura.

Hierarquia de Entidades

Entidade	Tabela (JPA)	Descrição
UsuarioTipo	usuario_tipos	Tipo de usuario (ex.: CLIENTE, DONO_RESTAURANTE)
Usuario	usuarios	Usuario do sistema; referencia UsuarioTipo
Restaurante	restaurantes	Estabelecimento; requer dono do tipo DONO_RESTAURANTE
Menuitem	menu_items	Item do cardapio vinculado a um Restaurante

Campos — UsuarioTipo

Campo Java	Coluna SQL	Tipo	Restricoes
id	id	BIGINT	PK, AUTO_INCREMENT
tipo	tipo	VARCHAR(255)	NOT NULL, UNIQUE

Campos — Usuario

Campo Java	Coluna SQL	Tipo	Restricoes
id	id	BIGINT	PK, AUTO_INCREMENT
nome	nome	VARCHAR(255)	NOT NULL
email	email	VARCHAR(255)	NOT NULL, UNIQUE
senha	senha	VARCHAR(255)	NOT NULL
usuarioTipoid	usuario_tipo_id	BIGINT	FK -> usuario_tipos.id

Campos — Restaurante

Campo Java	Coluna SQL	Tipo	Restricoes
id	id	BIGINT	PK, AUTO_INCREMENT
nome	nome	VARCHAR(255)	NOT NULL
endereço	endereço	VARCHAR(255)	NOT NULL
tipoCozinha	tipo_cozinha	VARCHAR(255)	NOT NULL
horarioFuncionamento	horario_funcionamento	VARCHAR(255)	NOT NULL

Campo Java	Coluna SQL	Tipo	Restricoes
donoId	dono_id	BIGINT	FK -> usuarios.id (tipo DONO_RESTAURANTE)

Campos — MenuItem

Campo Java	Coluna SQL	Tipo	Restricoes
id	id	BIGINT	PK, AUTO_INCREMENT
nome	nome	VARCHAR(255)	NOT NULL
descricao	descricao	TEXT	NOT NULL
preco	preco	DECIMAL(10,2)	NOT NULL
disponivel	disponivel	BOOLEAN	Consumo apenas no local
caminhoFoto	caminho_foto	VARCHAR(500)	Caminho/URL da foto do prato
restaurantId	restaurante_id	BIGINT	FK -> restaurantes.id

3. Endpoints Disponíveis

Todos os endpoints seguem o padrão REST. A API expõe quatro recursos: **tipos de usuário**, **usuários**, **restaurantes** e **itens do cardápio**.

3.1 Tipos de Usuário — /usuarios-tipos

Metodo	Rota	Status	Descrição
POST	/usuarios-tipos	201	Cria um tipo de usuário
GET	/usuarios-tipos	200	Lista todos os tipos
GET	/usuarios-tipos/{id}	200	Busca tipo por ID
PUT	/usuarios-tipos/{id}	200	Atualiza o nome do tipo
DELETE	/usuarios-tipos/{id}	204	Remove um tipo

3.2 Usuários — /usuarios

Metodo	Rota	Status	Descrição
POST	/usuarios	201	Cria um usuário (nome, email, senha, usuarioTipoid)
GET	/usuarios	200	Lista todos os usuários
GET	/usuarios/{id}	200	Busca usuário por ID
PUT	/usuarios/{id}	200	Atualiza o tipo do usuário
DELETE	/usuarios/{id}	204	Remove um usuário

3.3 Restaurantes — /restaurantes

Metodo	Rota	Status	Descrição
POST	/restaurantes	201	Cria restaurante (nome, endereço, tipoCozinha, horarioFuncionamento, donoid)
GET	/restaurantes	200	Lista todos os restaurantes
GET	/restaurantes/{id}	200	Busca restaurante por ID
PUT	/restaurantes/{id}	200	Atualiza dados do restaurante
DELETE	/restaurantes/{id}	204	Remove um restaurante

Regra de negócio: donoid deve referenciar um usuário existente cadastrado como **DONO_RESTAURANTE** (usuarioTipoid = 2). Caso contrário, a API retorna 400 Bad Request.

3.4 Itens do Cardápio — /menu-items

Metodo	Rota	Status	Descrição
POST	/menu-items	201	Cria item (nome, descricao, preco, disponivel, caminhoFoto, restaurantId)
GET	/menu-items	200	Lista todos os itens
GET	/menu-items/restaurant/{restaurantId}	200	Lista itens de um restaurante
GET	/menu-items/{id}	200	Busca item por ID
PUT	/menu-items/{id}	200	Atualiza item
DELETE	/menu-items/{id}	204	Remove item

3.5 Tratamento de Erros

Erros são retornados em formato padronizado { "status", "mensagem", "timestamp" }:

Exceção	Status HTTP
EntidadeNaoEncontradaException	404 Not Found
EntidadeJaExisteException	409 Conflict
RegraDeNegocioException (e subclasses, ex. dono invalido)	400 Bad Request
Erro não mapeado	500 Internal Server Error

Exemplo 1 — POST /restaurantes (Criar Restaurante)

Requisição:

```
{
  "nome": "Sushi do Chef",
  "endereco": "Av. Paulista, 1234, Sao Paulo - SP",
  "tipoCozinha": "Japonesa",
  "horarioFuncionamento": "Ter-Dom 12h-23h",
  "donoId": 1
}
```

Resposta de sucesso (201 Created):

```
{
  "id": 1,
  "nome": "Sushi do Chef",
  "endereco": "Av. Paulista, 1234, Sao Paulo - SP",
  "tipoCozinha": "Japonesa",
  "horarioFuncionamento": "Ter-Dom 12h-23h",
  "donoId": 1
}
```

Resposta de erro — dono invalido (400 Bad Request):

```
{
  "status": 400,
  "mensagem": "O usuario informado não é do tipo DONO_RESTAURANTE",
  "timestamp": "2026-07-14T10:30:00"
```

```
}
```

Exemplo 2 — POST /menu-items (Criar Item do Cardapio)

Requisicao:

```
{
  "nome": "Temaki Salmao",
  "descricao": "Temaki com salmao fresco, cream cheese e cebolinha",
  "preco": 28.90,
  "disponivel": true,
  "caminhoFoto": "/fotos/temaki-salmao.jpg",
  "restauranteId": 1
}
```

Resposta de sucesso (201 Created):

```
{
  "id": 1,
  "nome": "Temaki Salmao",
  "descricao": "Temaki com salmao fresco, cream cheese e cebolinha",
  "preco": 28.90,
  "disponivel": true,
  "caminhoFoto": "/fotos/temaki-salmao.jpg",
  "restauranteId": 1
}
```

Exemplo 3 — POST /usuarios-tipos (Criar Tipo de Usuario)

Requisicao:

```
{ "tipo": "DONO_RESTAURANTE" }
```

Resposta de sucesso (201 Created):

```
{ "id": 2, "tipo": "DONO_RESTAURANTE" }
```

Resposta de erro — tipo ja existe (409 Conflict):

```
{
  "status": 409,
  "mensagem": "Tipo de usuario ja cadastrado",
  "timestamp": "2026-07-14T10:30:00"
}
```

Exemplo 4 — GET /menu-items/restaurante/{restauranteId}

Requisicao: GET http://localhost:8080/menu-items/restaurante/1

Resposta de sucesso (200 OK):

```
[
  {
    "id": 1,
    "nome": "Temaki Salmao",
    "descricao": "Temaki com salmao fresco, cream cheese e cebolinha",
    "preco": 28.90,
    "disponivel": true,
    "caminhoFoto": "/fotos/temaki-salmao.jpg",
    "restauranteId": 1
  }
]
```

]

4. Colecao Postman

A colecao **Restaurantes Clean Arch.postman_collection.json** (Collection v2.1) esta inclusa no repositorio e cobre o CRUD completo dos quatro recursos. A variavel *base_url* ja aponta para **http://localhost:8080**.

As pastas seguem a ordem de dependencia entre os recursos: *usuarios-tipos* → *usuarios* → *restaurantes* → *menu-items* → *cleanup*. Os IDs sao encadeados automaticamente via variaveis de colecao — nao e necessario editar IDs manualmente entre requests.

Cenario	Metodo	Status Esperado
Criar tipo de usuario CLIENTE	POST	201 Created
Criar tipo de usuario DONO_RESTAURANTE	POST	201 Created
Criar tipo duplicado	POST	409 Conflict
Criar usuario valido	POST	201 Created
Criar usuario — email duplicado	POST	409 Conflict
Buscar usuario por ID — sucesso	GET	200 OK
Buscar usuario por ID — nao encontrado	GET	404 Not Found
Criar restaurante valido	POST	201 Created
Criar restaurante — dono invalido	POST	400 Bad Request
Listar restaurantes	GET	200 OK
Buscar restaurante por ID	GET	200 OK
Atualizar restaurante	PUT	200 OK
Excluir restaurante	DELETE	204 No Content
Criar item do cardapio	POST	201 Created
Listar itens por restaurante	GET	200 OK
Atualizar item do cardapio	PUT	200 OK
Excluir item do cardapio	DELETE	204 No Content
Cleanup geral	DELETE	204 No Content

5. Estrutura do Banco de Dados

Banco utilizado: **PostgreSQL 17** executando em container Docker. As tabelas são criadas automaticamente pelo Hibernate com *ddl-auto: update*. Não há herança JPA — cada entidade possui sua própria tabela com chave primária própria e foreign keys explícitas.

Tabela: usuario_tipos

Coluna	Tipo SQL	Restrições	Descrição
id	BIGINT	PK, AUTO_INCREMENT	Identificador único
tipo	VARCHAR(255)	NOT NULL, UNIQUE	Nome do tipo (ex.: CLIENTE)

Tabela: usuarios

Coluna	Tipo SQL	Restrições	Descrição
id	BIGINT	PK, AUTO_INCREMENT	Identificador único
nome	VARCHAR(255)	NOT NULL	Nome completo
email	VARCHAR(255)	NOT NULL, UNIQUE	E-mail único
senha	VARCHAR(255)	NOT NULL	Senha
usuario_tipo_id	BIGINT	FK -> usuario_tipos.id	Tipo do usuário

Tabela: restaurantes

Coluna	Tipo SQL	Restrições	Descrição
id	BIGINT	PK, AUTO_INCREMENT	Identificador único
nome	VARCHAR(255)	NOT NULL	Nome do restaurante
endereco	VARCHAR(255)	NOT NULL	Endereço completo
tipo_cozinha	VARCHAR(255)	NOT NULL	Ex.: Italiana, Japonesa
horario_funcionamento	VARCHAR(255)	NOT NULL	Ex.: Seg-Sex 11h-22h
dono_id	BIGINT	FK -> usuarios.id	Dono do restaurante

Tabela: menu_items

Coluna	Tipo SQL	Restrições	Descrição
id	BIGINT	PK, AUTO_INCREMENT	Identificador único
nome	VARCHAR(255)	NOT NULL	Nome do prato
descricao	TEXT	NOT NULL	Descrição detalhada
preco	DECIMAL(10,2)	NOT NULL	Preço do item

Coluna	Tipo SQL	Restricoes	Descricao
disponivel	BOOLEAN	NOT NULL	Apenas no local
caminho_foto	VARCHAR(500)	-	Caminho/URL da foto
restaurante_id	BIGINT	FK -> restaurantes.id	Restaurante do item

6. Execucao com Docker Compose

A aplicacao e o banco de dados rodam em containers Docker orquestrados pelo Docker Compose, dentro de uma rede dedicada. Os containers se comunicam pelo nome do servico, nao por localhost.

Variaveis de Ambiente

Variavel	Valor	Descricao
SPRING_DATASOURCE_URL	jdbc:postgresql://postgres:5432/restaurntes_clean_arch	URL de conexao com banco
SPRING_DATASOURCE_USERNAME	admin	Usuario do banco
SPRING_DATASOURCE_PASSWORD	admin123	Senha do banco
POSTGRES_DB	restaurantes_clean_arch	Nome do banco criado

Passo a Passo para Execucao

#	Etapas	Comando / Acao
1	Pre-requisito	Ter Docker e Docker Compose instalados
2	Clonar o repositorio	git clone
3	Entrar na pasta	cd restaurantes-clean-arch
4	Subir os containers	docker compose up --build
5	Verificar containers	docker compose ps
6	Acessar a API	http://localhost:8080
7	Parar os containers	docker compose down

Execucao Local com Maven

Pre-requisitos: Java 17 e Maven (ou o wrapper `./mvnw`) com um Postgres acessivel em localhost:5432. As credenciais devem estar configuradas em `src/main/resources/application.properties`.

```
spring.datasource.url=jdbc:postgresql://localhost:5432/restaurantes_clean_arch
spring.datasource.username=admin
spring.datasource.password=admin123

# Executar:
./mvnw spring-boot:run
```

7. Testes e Cobertura

A suite de testes é dividida em **testes unitários** e **testes de integração**. O gate de cobertura mínima de **80% de linhas** é aplicado automaticamente via JaCoCo durante o *mvn verify*. A cobertura atual está acima de 90%.

Testes Unitários — core/domain e core/usecase

Cobrem invariantes de domínio (construtor com validações) e regras de negócio dos casos de uso, com **Mockito** simulando os Gateways.

Classe de Teste	Cenários Cobertos	Resultado
UsuarioTipoTest	Criacao valida; tipo vazio lancando excecao	PASS
UsuarioTest	Criacao com todos os campos; campos obrigatorios nulos	PASS
RestauranteTest	Criacao valida; nome vazio; donold nulo	PASS
MenuItemTest	Criacao com preco valido; preco negativo lancando excecao	PASS
CriarUsuarioTipoImplTest	Cria tipo; rejeita tipo duplicado	PASS
CriarUsuarioImplTest	Cria usuario; rejeita email duplicado	PASS
CriarRestauranteImplTest	Cria restaurante; rejeita dono invalido	PASS
CriarMenuItemImplTest	Cria item; rejeita restaurante inexistente	PASS
ListarRestaurantesImplTest	Retorna lista; retorna lista vazia	PASS
BuscarMenuItemImplTest	Retorna item por ID; lanca 404 para ID inexistente	PASS
AtualizarRestauranteImplTest	Atualiza com dados validos; lanca 404 se nao encontrado	PASS
DeletarMenuItemImplTest	Deleta item existente; lanca 404 se nao encontrado	PASS

Testes de Integracao — infra/web/*ControllerIT.java

Sobem o contexto Spring completo (@SpringBootTest) e exercitam os endpoints via **TestRestTemplate** contra um Postgres real. Cobrem o CRUD ponta-a-ponta de todos os quatro recursos.

Classe de Teste	Endpoints Testados	Resultado
UsuarioTipoControllerIT	POST / GET / PUT / DELETE /usuarios-tipos	PASS
UsuarioControllerIT	POST / GET / PUT / DELETE /usuarios	PASS
RestauranteControllerIT	POST / GET / PUT / DELETE /restaurantes	PASS
MenuItemControllerIT	POST / GET / PUT / DELETE /menu-items	PASS

Executando a Suite Completa

```
# Sobe o banco usado pelos testes de integracao
docker compose up -d postgres

# Executa todos os testes e aplica o gate de cobertura (minimo 80%)
./mvnw verify

# Relatorio HTML de cobertura gerado em:
# target/site/jacoco/index.html
```

Resumo de Cobertura

Metrica	Minimo Exigido	Cobertura Atual
Cobertura de Linhas	80%	> 90%
Testes Unitarios	-	Dominio + Casos de Uso
Testes de Integracao	-	Ponta-a-ponta via TestRestTemplate
Gate de build (JaCoCo)	80%	Configurado — build falha abaixo do minimo

8. Mapeamento de Requisitos da Fase 2

Tabela de rastreabilidade entre os requisitos do Tech Challenge Fase 2 e as implementações entregues neste projeto.

Requisito do PDF	Como foi implementado
Tipo de usuário — CRUD com campo Nome do Tipo e associação com usuário	UsuarioTipo e entidade de domínio; CRUD completo em /usuarios-tipos. Associação via usuarioTipoid em Usuario, atualizável em PUT /usuarios/{id}.
Cadastro de restaurante — nome, endereço, tipo de cozinha, horário, dono	CRUD completo em /restaurantes. donoid referencia um Usuario existente; a criação válida que o usuário é do tipo DONO_RESTAURANTE.
Cadastro de itens do cardápio — nome, descrição, preço, disponibilidade no local, caminho da foto	CRUD completo em /menu-items com todos os campos exigidos e vínculo a um restaurantid.
Qualidade do código / práticas Spring Boot	Camadas isoladas, injeção de dependência explícita, DTOs de entrada/saída, tratamento centralizado de erros.
Documentação do projeto	Este documento (arquitetura, endpoints, setup) e o README.md no repositório.
Collections para teste	Restaurants Clean Arch.postman_collection.json cobre o CRUD completo dos 4 recursos com test scripts.
Configuração Docker Compose	docker-compose.yml sobre a aplicação Java e o Postgres em rede dedicada.
Repositório de código aberto	Projeto versionado no GitHub.
Clean Architecture	Organização em core (domain, usecase, controller, gateway) e infra (web, database, config).
Cobertura de testes de 80%	JaCoCo configurado com gate de 80%. Testes unitários e de integração. Cobertura atual acima de 90%.
Video demonstrativo	Entregue separadamente do repositório, com duração de aproximadamente 5 minutos.